

Abstract

Web applications are critical components of modern digital systems, yet their reliance on user input exposes them to numerous security risks. Although Web Application Firewalls (WAFs) are widely deployed to protect against common web attacks, real-world incidents show that such defenses are not always sufficient. This report investigates how input-based attacks, including Cross-Site Scripting (XSS) and Server-Side Request Forgery (SSRF), can be adapted to bypass WAF protections and exploit system vulnerabilities.

The study begins with a review of ethical hacking principles and WAF architecture, highlighting their strengths and limitations. A structured testing methodology was applied in a controlled lab environment using OWASP Juice Shop, encompassing reconnaissance, input discovery, scanning, vulnerability identification, evasion, exploitation, and post-exploitation analysis. Various attack payloads were tested to evaluate WAF effectiveness, and adaptive techniques were applied to bypass filtering mechanisms.

A real-world case study of the 2018 British Airways data breach is examined to illustrate the impact of input-based attacks on live systems. Additionally, the legal and ethical context is considered, specifically in relation to the Electronic Transaction Act 2063 (Nepal), to ensure that ethical hacking practices remain within regulatory boundaries.

The findings demonstrate that WAFs provide an important layer of defense but can be circumvented through adaptive payloads and inconsistent input handling. The report concludes by emphasizing the need for a layered security approach, rigorous testing procedures, and the implementation of context-aware and normalized filtering to strengthen web application defenses.

1. Introduction

1.1 Introduction to Ethical Hacking

Security mechanisms aim to protect systems, yet real-world incidents consistently demonstrate that these defences are not always effective. The gap between expected and actual system security highlights the need for practical, evaluation-driven testing under realistic attack conditions (Stallings, 2018).

Ethical hacking involves systematic identification, analysis, and remediation of security vulnerabilities in computer systems, networks, and applications under authorized and controlled conditions. Certified professionals, known as white-hat hackers, replicate the tools, techniques, and methodologies of malicious attackers but with the goal of strengthening security rather than exploiting it (Coursera Staff, 2026).

These domains are summarized in the figure below (Sharma, 2026):



Figure 1: Types of Ethical Hacking

[Click here for Advantages and limitations of Ethical Hacking](#)

1.2 Introduction to Web Application Firewalls (WAF)

Modern web applications face a constant barrage of malicious traffic, from automated bots to sophisticated SQL injection attempts. To mitigate these threats, organizations deploy Web Application Firewalls (WAFs), positioned strategically to monitor, filter, and block harmful HTTP requests (cloudflare, 2026).

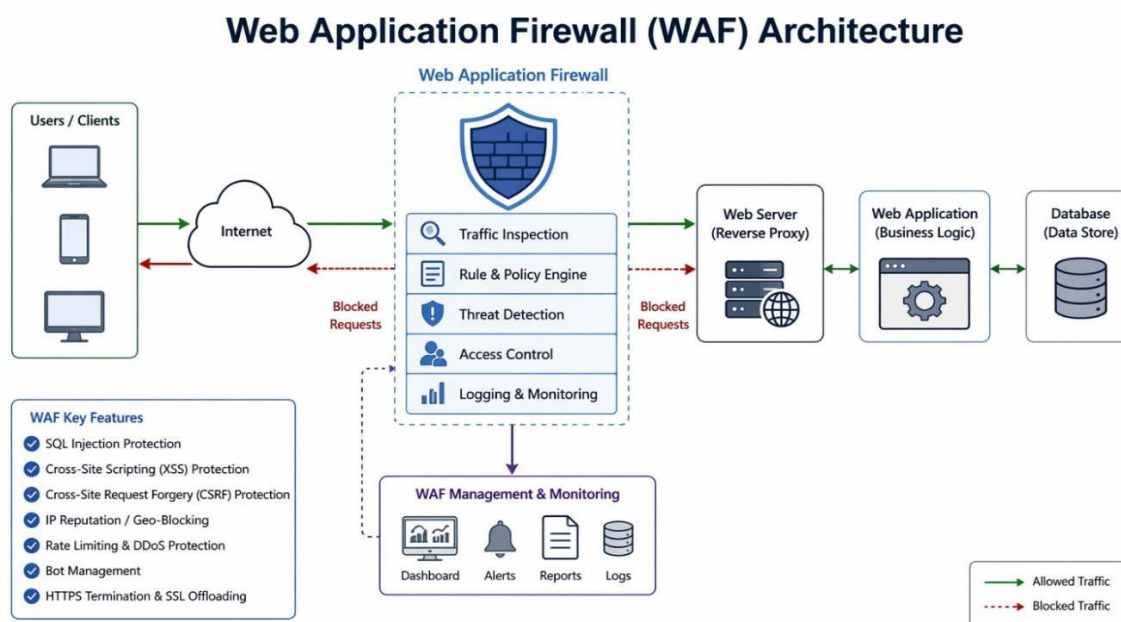


Figure 2: Web Application Firewall (WAF) Architecture

WAFs inspect different parts of a request, including URLs, headers, and input parameters, and apply predefined rules or behavioral analysis to determine whether the request should be allowed or blocked. They are commonly deployed as reverse proxies, allowing them to intercept traffic before it reaches the web server (Finn, 2026).

By inspecting URLs, headers, cookies, and input parameters, WAFs use either predefined rulesets or behavioral analysis to decide which traffic is safe. Deployment options include (indusface, 2026):

DEPLOYMENT OPTIONS FOR WAF

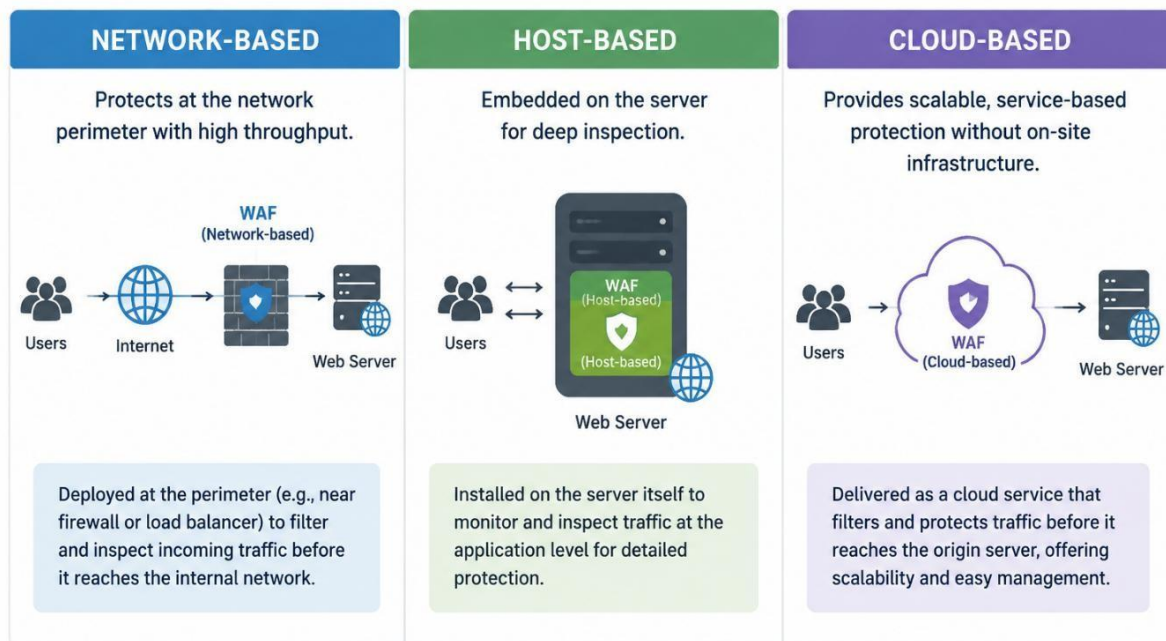


Figure 3 Types of WAF

WAFs often operate as reverse proxies, intercepting traffic before it reaches the web server to prevent malicious requests from impacting the application. [Click here for advantages and limitations of WAF.](#)

1.3 Phases of Ethical Hacking

Ethical hacking follows a structured methodology that simulates real-world attacks under authorized and controlled conditions. These phases help security professionals identify vulnerabilities, evaluate system defences, and improve organizational security posture (geeksforgeeks, 2026).



Figure 4: Phases of Ethical Hacking

[Click here for elaborated phases of ethical hacking](#)

1.4 Aims and Objectives

1.4.1 Aims

To demonstrate how input-based attacks, such as XSS and SSRF, can bypass Web Application Firewall protections and to evaluate the effectiveness of these defenses in a controlled web application environment.

1.4.2 Objectives

- To identify input points and potential attack surfaces within a web application
- To perform reconnaissance, scanning, and vulnerability identification against user-controlled inputs
- To test system responses using baseline XSS and SSRF payloads to detect security controls
- To apply and evaluate payload modification and WAF evasion techniques
- To demonstrate successful exploitation after bypassing WAF protections
- To analyze post-exploitation impact, including internal API access through SSRF techniques
- To assess how attack effectiveness varies across different application contexts
- To evaluate the effectiveness and limitations of signature-based WAF mechanisms
- To support findings using relevant real-world case studies
- To examine the legal and ethical implications of ethical hacking in relation to the Electronic Transaction Act 2063 (Nepal)
- To propose recommendations for strengthening web application security and WAF filtering mechanisms

2. Background

2.1 Background of the Topic

Web applications rely heavily on user input for functions such as authentication, search, and data processing, making them vulnerable to security risks due to unpredictable input handling. Improper validation can introduce weaknesses that attackers may exploit.

To reduce these risks, developers use validation mechanisms and Web Application Firewalls (WAFs) that inspect and filter requests using rules, signatures, or behavioral analysis. However, inconsistent filtering across different application contexts can create exploitable gaps.

Attackers typically do not rely on a single payload; instead, they adapt input variations based on system responses, making web security a dynamic interaction between attack and defense. Techniques such as XSS and SSRF demonstrate how modified payloads can bypass filtering controls (Desai, 2026).

This study examines how such input-based attacks bypass filtering mechanisms in different contexts, using a controlled OWASP Juice Shop environment to safely evaluate WAF behavior and security impact.

[Click here for methodology for WAF Evasion](#)

3. Literature Review

3.1 Case Study: British Airways Data Breach (2018)

Overview

In 2018, British Airways experienced a significant data breach affecting approximately 380,000 customers, where sensitive payment information was compromised. The attack was attributed to a Magecart-style client-side web skimming technique, in which malicious JavaScript was injected into the airline's website. This code intercepted and exfiltrated customer data during the payment process (Khandelwal, 2026).

[Click here for detailed analysis](#)

4. Attack Demonstration

4.1 Reconnaissance

4.1.1 Application Environment Verification

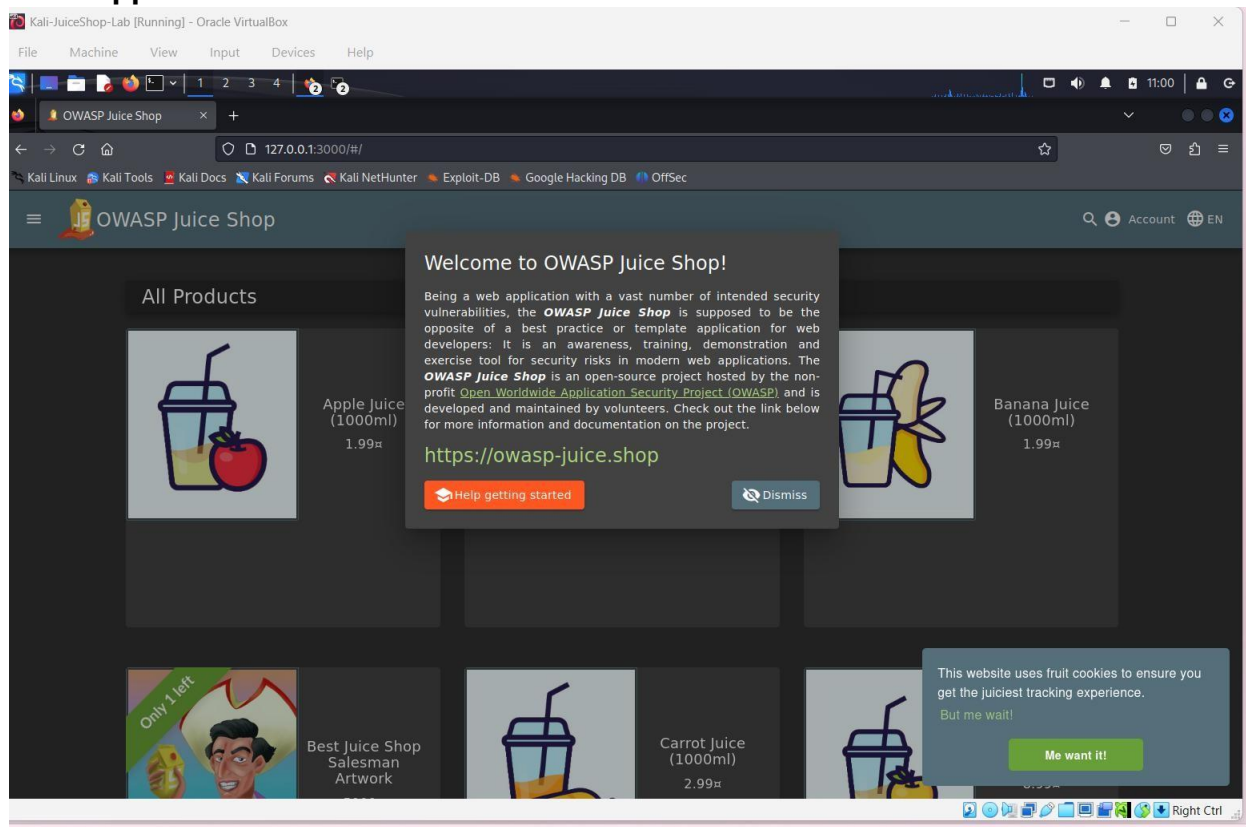


Figure 5 Reconnaissance: OWASP Juice Shop running locally at 127.0.0.1:3000

Field	Details
Recon Activity	Local application and navigation review
Purpose	Confirm that OWASP Juice Shop is running locally and ready for testing
Key Observation	Homepage displays product listings, search, account, language, and side navigation options
Relevance	Confirms controlled lab environment and identifies visible areas for further testing

Table 1: Demonstration Table 1

4.1.2 Homepage and Navigation Review

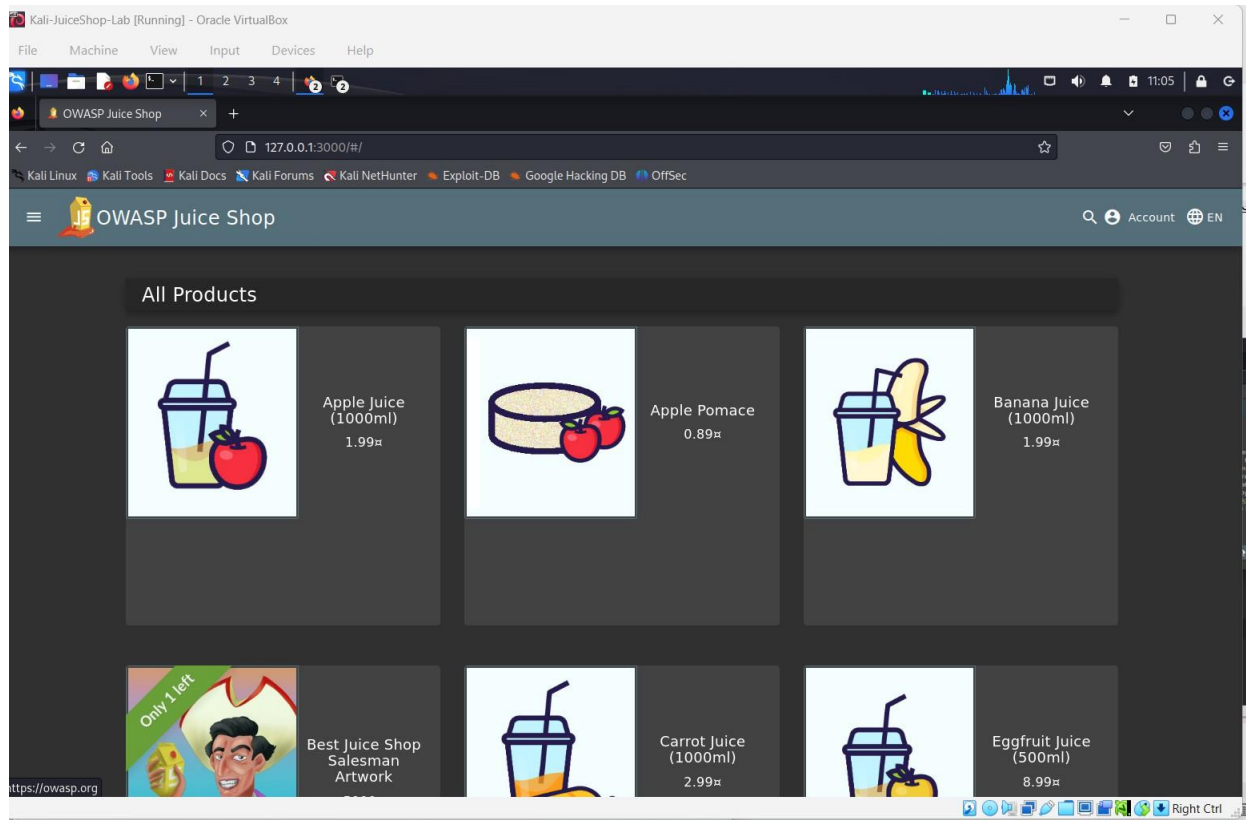


Figure 6 Reconnaissance: Juice Shop homepage and visible navigation

Field	Details
Recon Activity	Local application and navigation review
Purpose	To confirm that OWASP Juice Shop was running locally and to identify visible application features.
Key Observation	The homepage displayed product listings, search, account, language, and side navigation options.
Relevance	Confirms the controlled local lab environment and identifies visible areas for further testing.

Table 2: Demonstration Table 2

4.1.3 Input Discovery

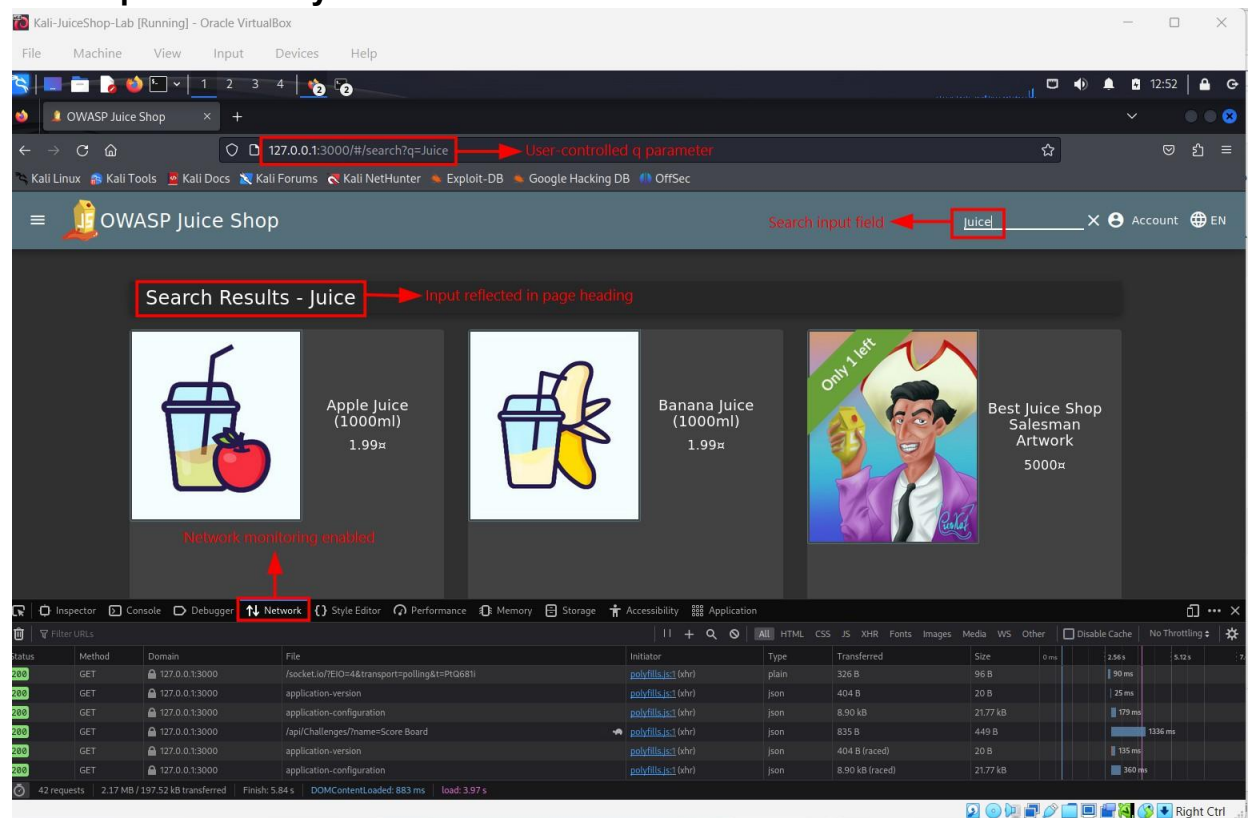


Figure 7 Reconnaissance: Search Input Discovery

Field	Details
Recon Activity	Search input discovery
Purpose	To identify a user-controlled input point suitable for payload testing.
Key Observation	The value juice appeared in the q URL parameter and was reflected in both the search field and the page heading.
Relevance	Establishes the search function as a suitable target for later XSS and WAF evasion testing.

Table 3: Demonstration Table 3

4.1.4 Network Traffic and Endpoint Inspection

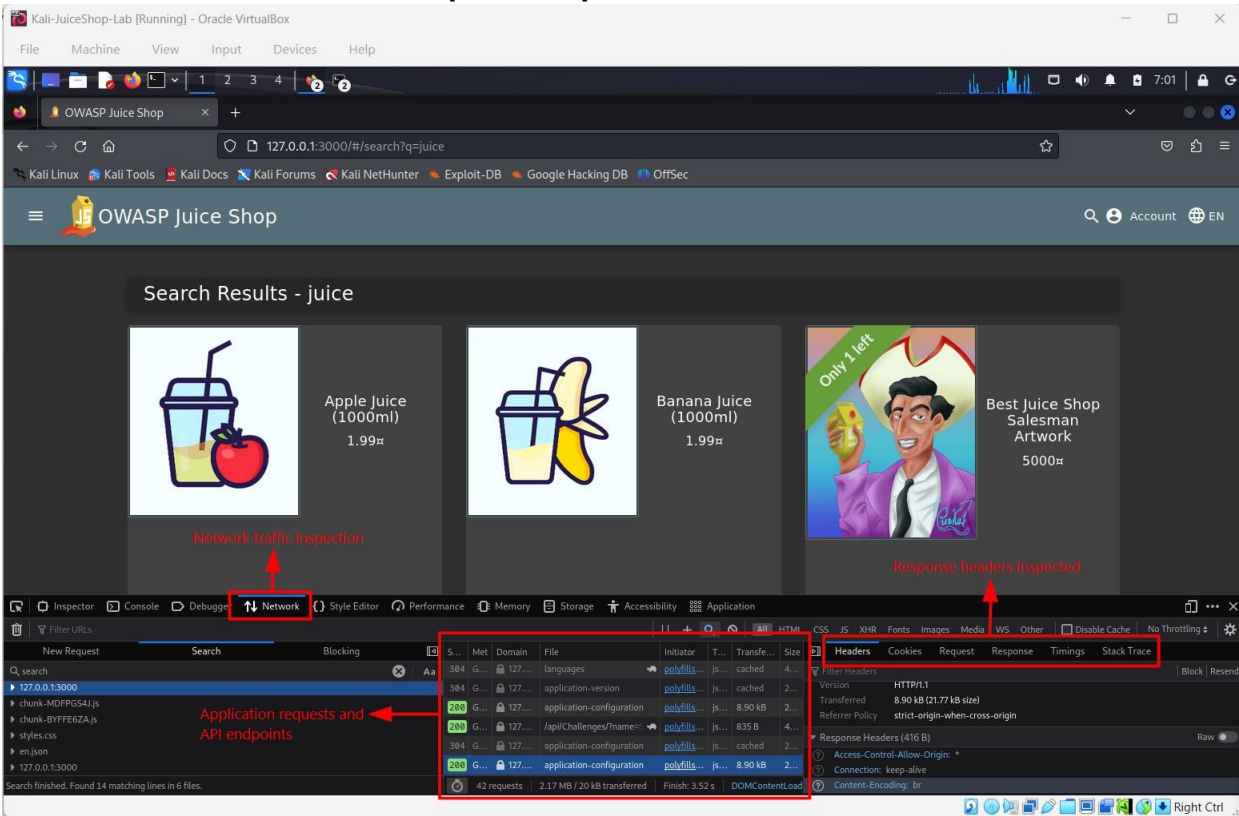


Figure 8 Reconnaissance: Network Traffic and Endpoint Inspection

Field	Details
Recon Activity	Network traffic and endpoint inspection
Purpose	To observe how the frontend communicates with backend resources during user interaction.
Key Observation	Developer Tools showed application requests, API-related endpoints, and response headers from the local server.
Relevance	Helps understand application request flow and supports later monitoring of payload behavior.

Table 4: Demonstration Table 4

4.2 Scanning and Enumeration

4.2.1 Filter Behavior Fingerprinting– Baseline XSS Testing

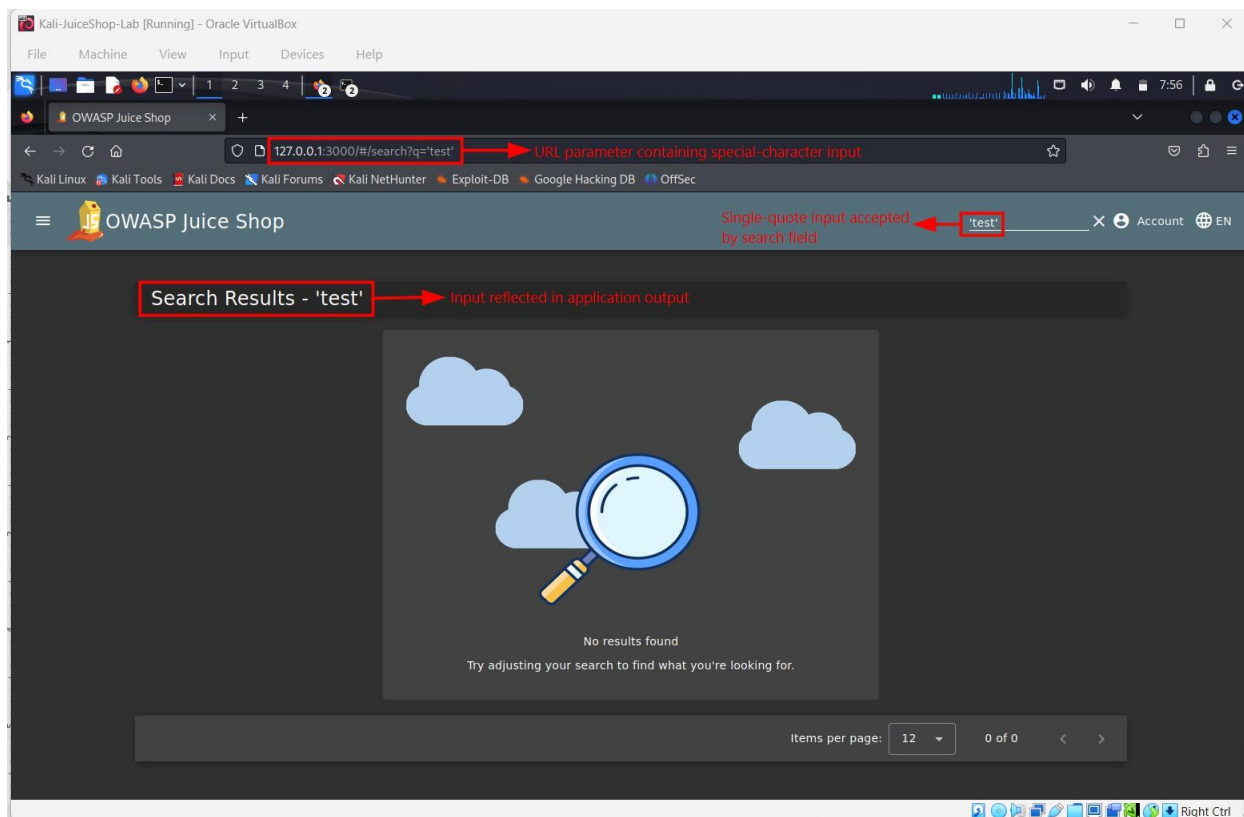


Figure 9 Scanning and Enumeration: Filter Behavior Fingerprinting

Field	Details
Activity	Special-character input fingerprinting
Purpose	To observe how the search function handles non-alphanumeric input before full payload testing.
Key Observation	The input 'test' was accepted through the q parameter, displayed in the search field, and reflected in the page heading.
Relevance	Shows that the search function accepts and reflects special-character input, making it suitable for deeper payload testing.

Table 5: Demonstration Table 5

4.3 Vulnerability Identification

4.3.1 Vulnerability Identification – Baseline XSS Testing

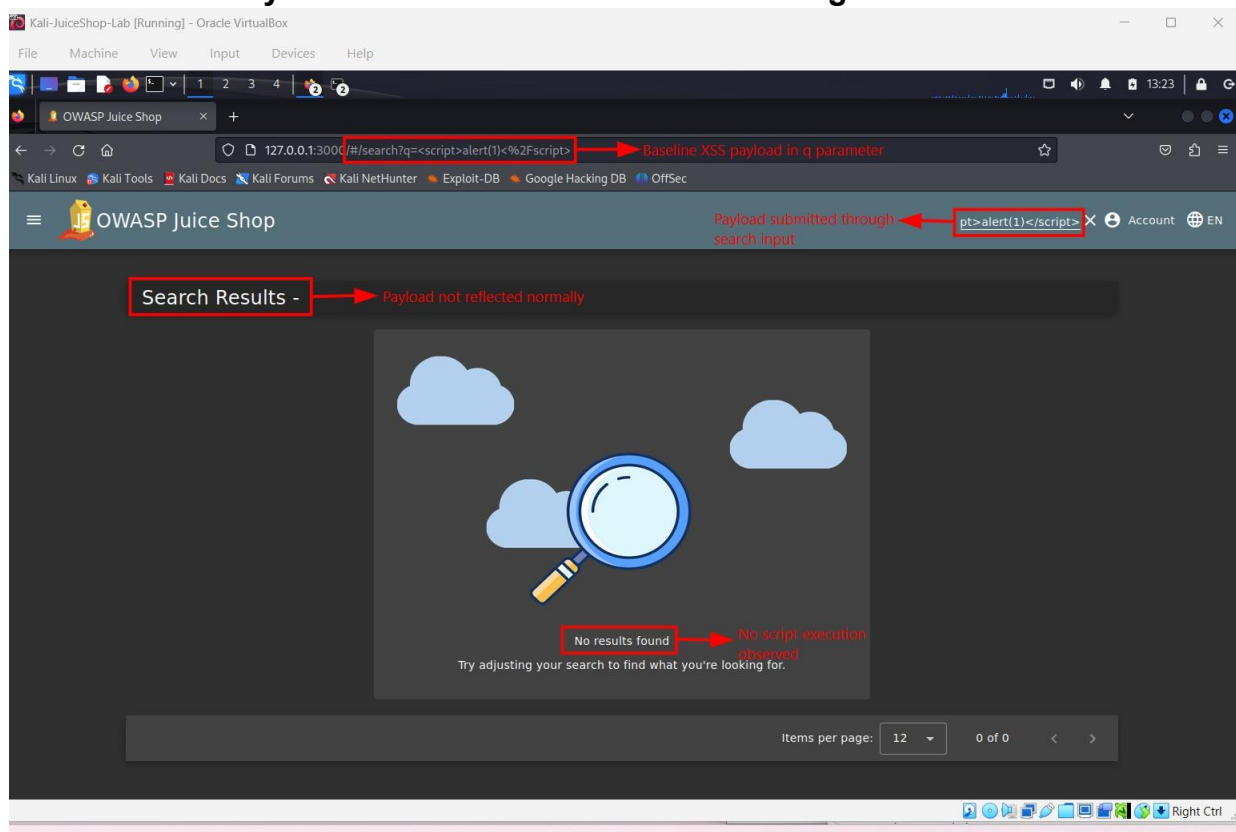


Figure 10 Vulnerability Identification: Baseline XSS Payload Evaluation

Field	Details
Activity	Baseline XSS payload submission
Purpose	To test whether the application executes a standard <code><script>alert(1)</script></code> payload in the search input
Key Observation	The payload <code><script>alert(1)</script></code> appeared in the search results heading but was not executed
Relevance	Establishes a baseline for payload handling and sets up further evasion tests

Table 6: Demonstration Table 6

4.3.2 Vulnerability Identification – Modified XSS Testing

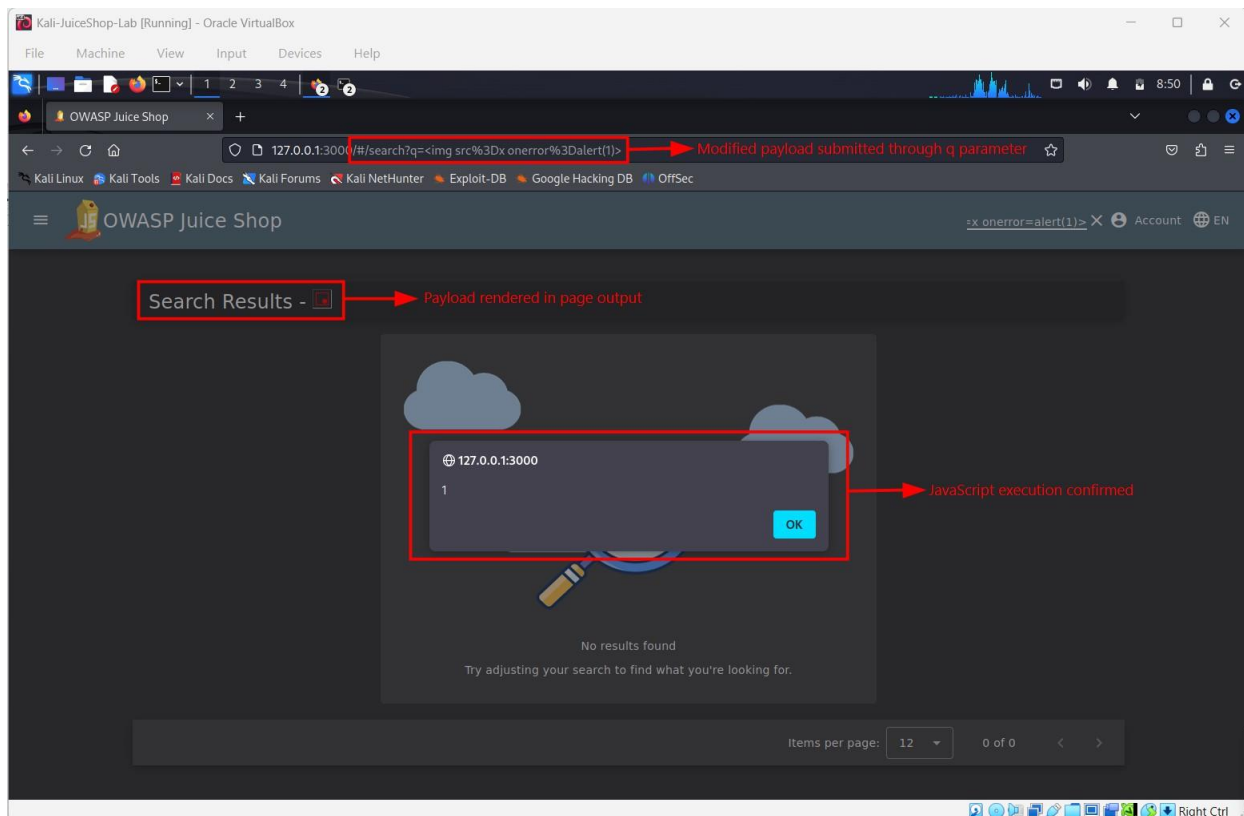


Figure 11 Vulnerability Identification: Successful XSS Execution

Field	Details
Test Activity	Modified XSS payload execution
Target Function	Search input (q parameter)
Payload Used	`<img src=x onerror=alert(1)>`
Result	Executed successfully
Key Observation	The payload rendered in the application output and triggered a JavaScript alert
Interpretation	The search function did not execute the direct script-tag payload, but it executed a modified event-handler payload

Table 7: Demonstration Table 7

4.3.3 Vulnerability Identification – Context-Aware XSS Testing

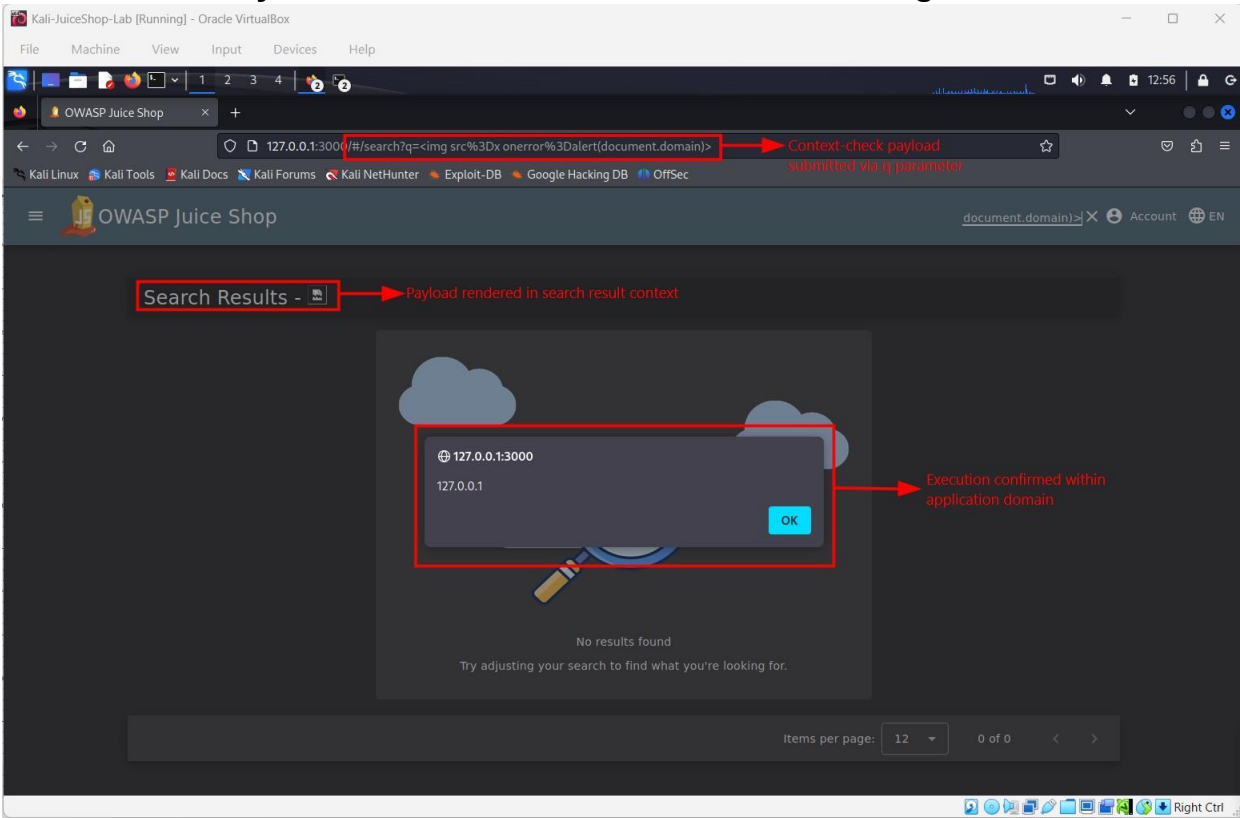


Figure 12 Vulnerability Identification: Page Context Confirmation

Field	Details
Test Activity	Page context confirmation
Target Function	Search function
Payload Used	
Result	Executed successfully
Key Observation	The alert displayed 127.0.0.1, confirming execution within the local Juice Shop page context.
Interpretation	The payload was interpreted as active client-side content and executed within the application domain.

Table 8: Demonstration Table 8

[Click here for comparison table](#)

4.3.4 SSRF Vulnerability Validation

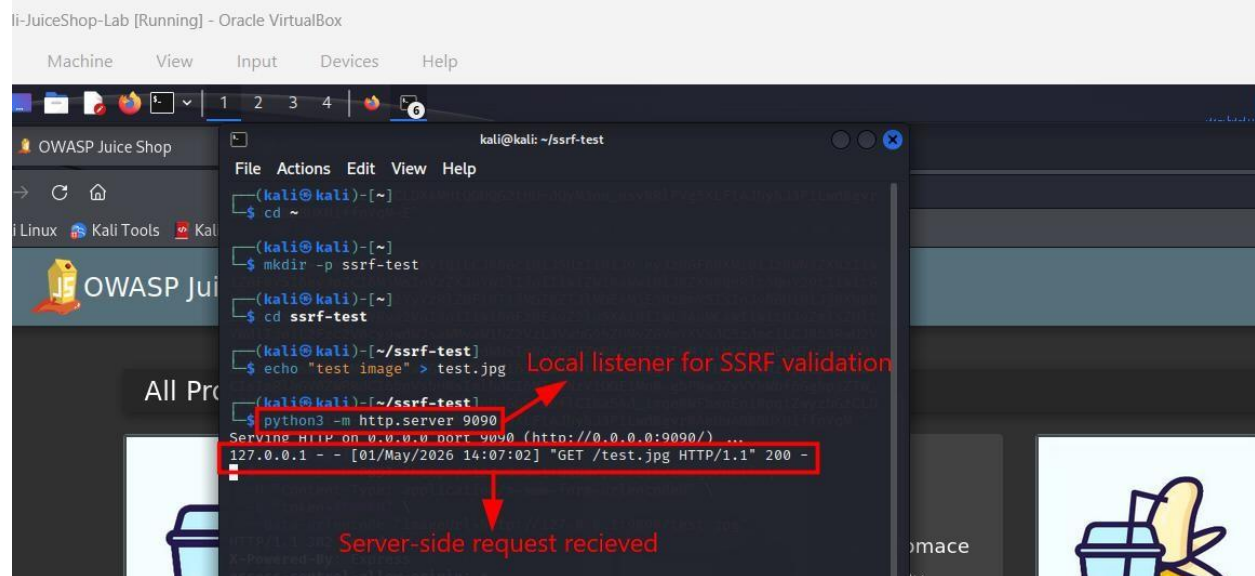


Figure 13: Vulnerability Identification – SSRF Validation

Field	Details
Test Activity	SSRF validation
Target Feature	Profile image URL upload
Supplied URL	http://127.0.0.1:9090/test.jpg
Listener	Python HTTP server on port 9090
Result	GET /test.jpg received
Interpretation	Juice Shop made a server-side request to the supplied URL, confirming SSRF behavior.

Table 9: Demonstration Table 9

4.4 Exploitation through WAF Evasion

This phase evaluated the effectiveness of the local WAF proxy against previously identified XSS and SSRF payloads. Standard payloads were first tested to observe filtering behavior, followed by modified payloads designed to evade signature-based detection mechanisms.

4.4.1 Baseline Payload Blocking (WAF Detection)

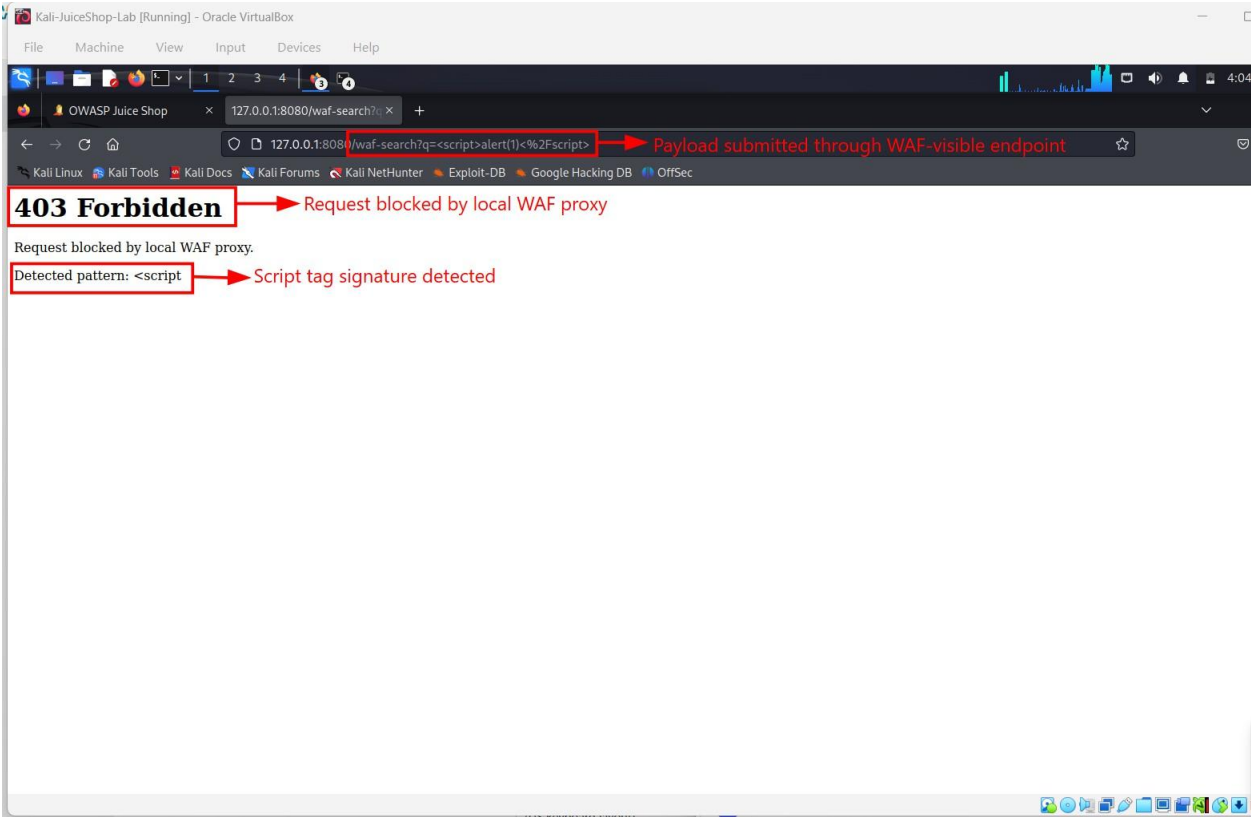


Figure 14: Exploitation through WAF Evasion – XSS Baseline Payload Blocked

Field	Details
Test Activity	Baseline payload block test
Target Path	/waf-search?q=
Payload Used	<script>alert(1)</script>
WAF Result	Blocked
Response	HTTP 403 Forbidden
Detected Pattern	<script>
Interpretation	The local WAF proxy detected and blocked a direct script-tag payload before forwarding it to Juice Shop.

Table 10: Demonstration Table 10

4.4.2 Event-Handler Payload Blocking

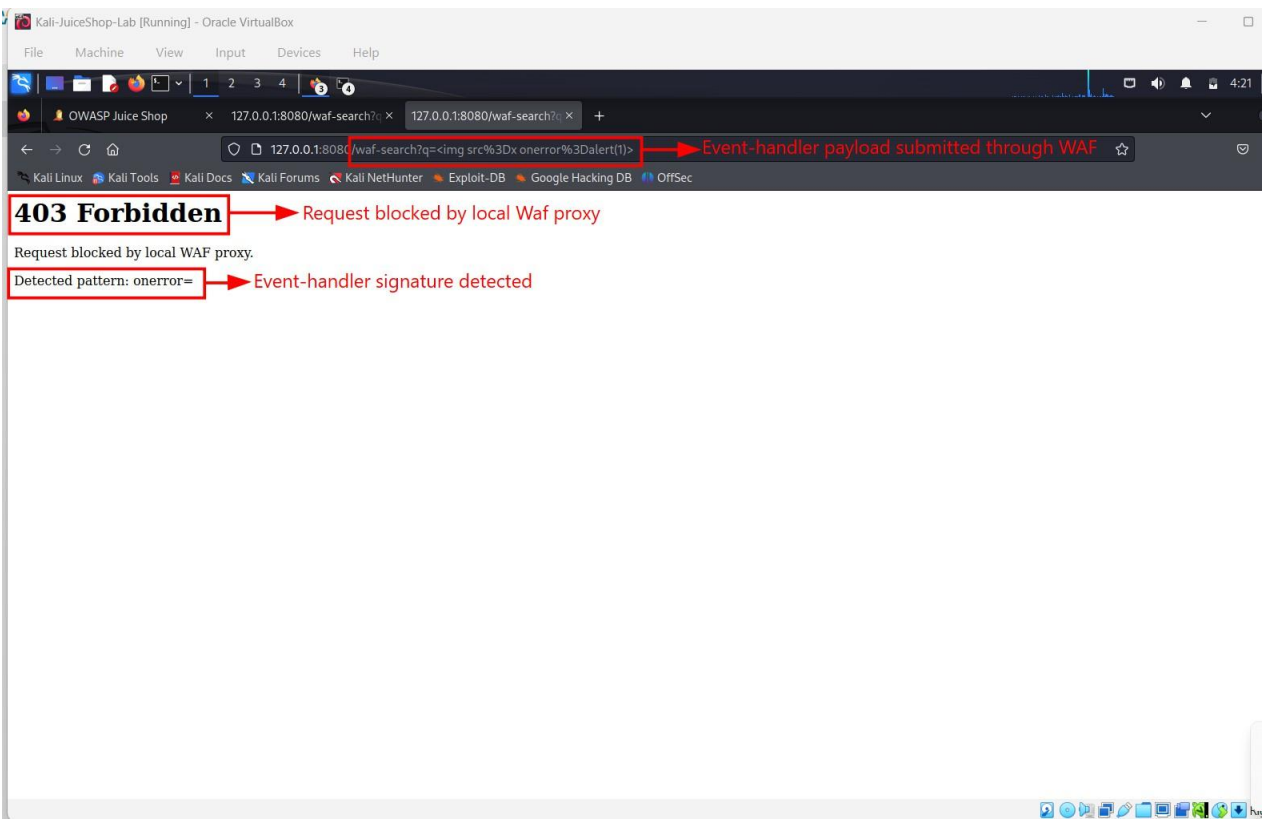


Figure 15: Exploitation through WAF Evasion – XSS Event-Handler Payload Blocked

Field	Details
Test Activity	Event-handler payload block test
Target Path	/waf-search?q=
Payload Used	
WAF Result	Blocked
Response	HTTP 403 Forbidden
Detected Pattern	onerror=
Interpretation	The WAF detected and blocked the previously successful image event-handler payload before it reached Juice Shop.

Table 11: Demonstration Table 11

4.4.3 WAF Evasion – Adapted Payload Execution

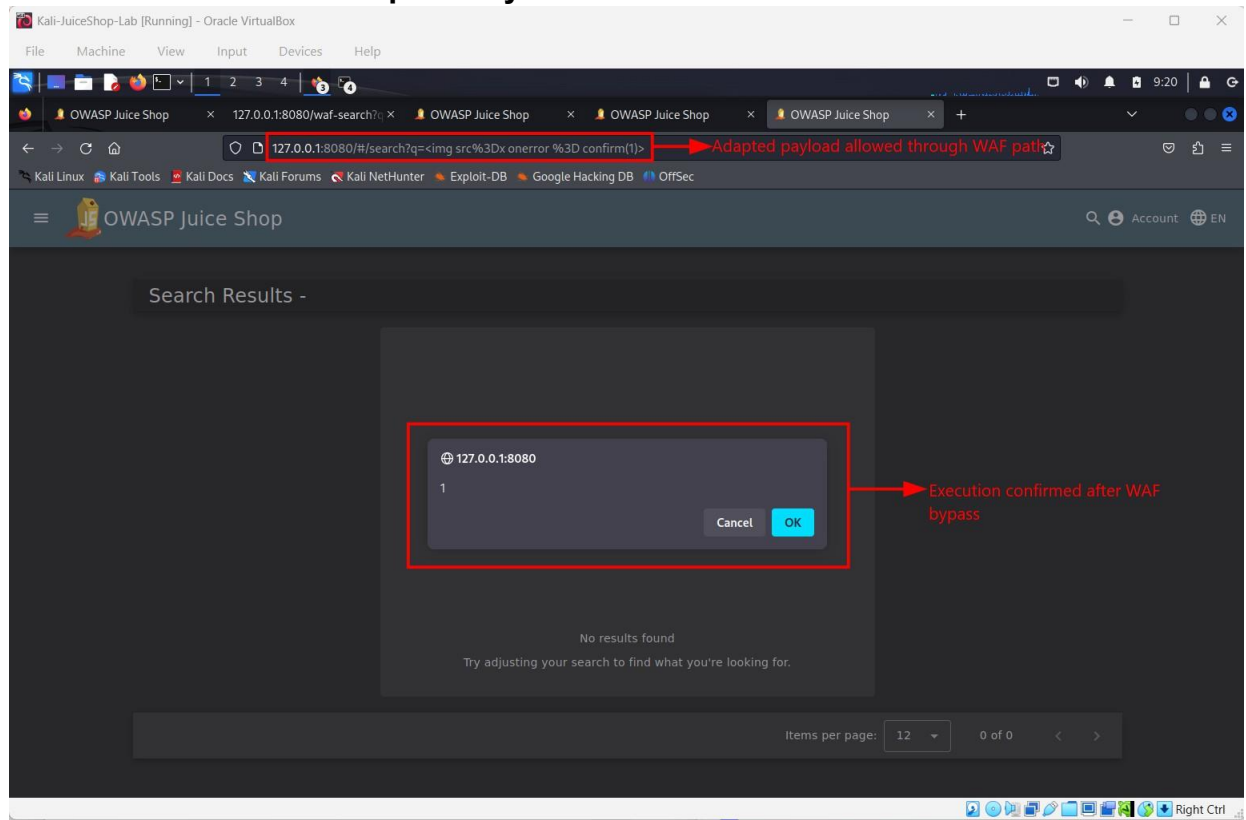


Figure 16: Exploitation through WAF Evasion – XSS Adapted Payload Bypass

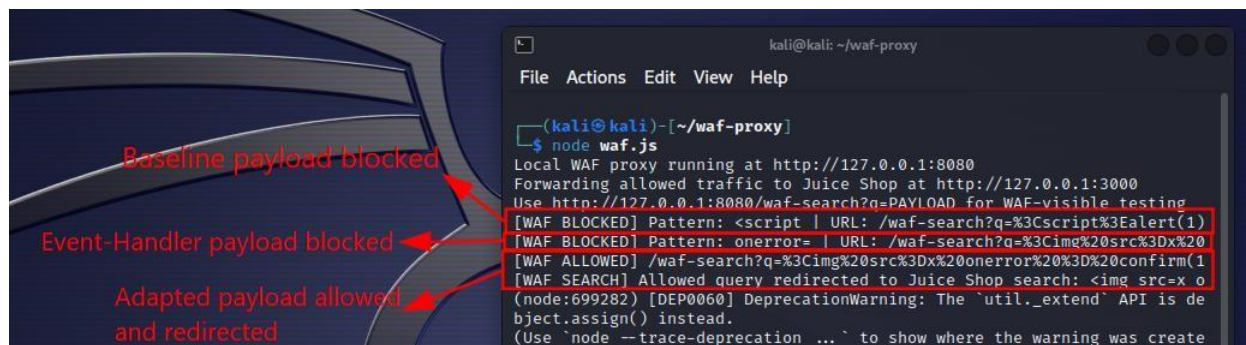


Figure 17: Exploitation through WAF Evasion: Successful XSS Payload Bypass

Field	Details
Test Activity	Adapted XSS payload execution and WAF console monitoring
Target Function / Path	Search input (q parameter) via WAF endpoint (<code>/waf-search?q=</code>)
Payload Used	<code></code>

WAF Result	Allowed
Response	Executed successfully in browser
Key Observation	Browser shows a confirm alert; WAF console shows blocked baseline and event-handler payloads, allowed adapted payload
Interpretation	The adapted payload bypassed WAF signature-based filtering, successfully executed, and confirms that WAF can be evaded with modified payloads

Table 12: Demonstration Table 12

4.4.4 SSRF WAF Blocking Test

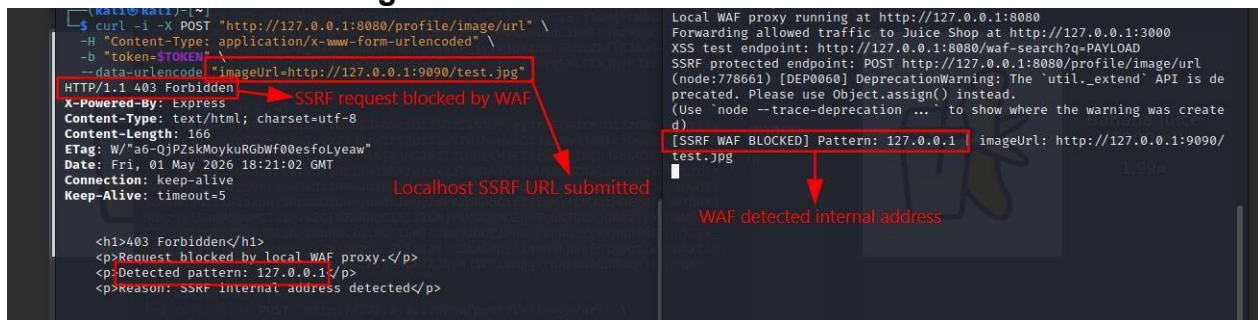


Figure 18: Exploitation through WAF Evasion – SSRF WAF Blocking Test

Field	Details
Test Activity	SSRF WAF blocking test
Target Endpoint	/profile/image/url
Submitted URL	http://127.0.0.1:9090/test.jpg
WAF Result	Blocked
Response	HTTP 403 Forbidden
Detection Pattern	127.0.0.1
Key Observation	The WAF identified the localhost SSRF request and blocked it before forwarding the request to Juice Shop
Interpretation	The WAF relied on direct localhost pattern detection to prevent server-side requests to internal resources

Table 13: Demonstration Table 13

4.4.5 SSRF WAF Evasion – Localhost Representation Bypass

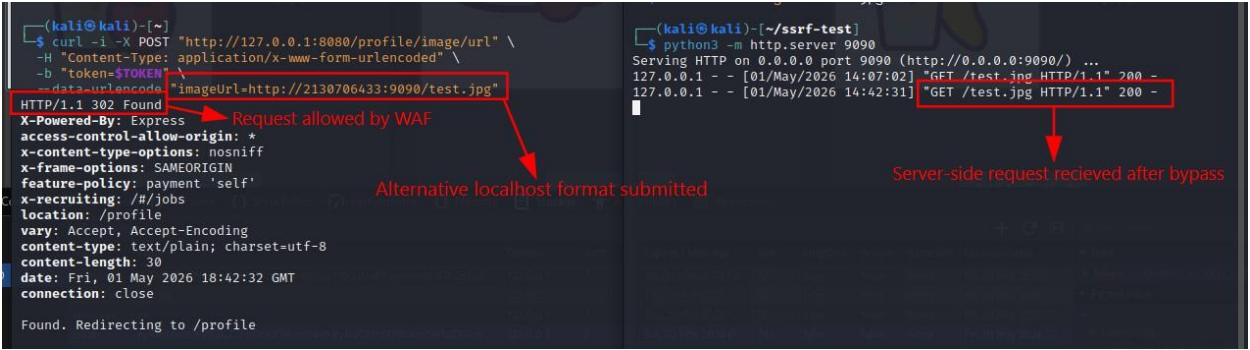


Figure 19:Exploitation through WAF Evasion – SSRF Localhost Representation Bypass

Field	Details
Test Activity	SSRF WAF bypass
Target Endpoint	/profile/image/url
Blocked URL	http://127.0.0.1:9090/test.jpg
Bypass URL	http://2130706433:9090/test.jpg
WAF Result	Allowed
Application Response	302 Found redirect to /profile
Listener Evidence	GET /test.jpg received
Evasion Technique	Alternative localhost representation
Interpretation	The WAF blocked the literal localhost string but failed to detect an alternative numeric representation that still resolved to localhost.

Table 14: Demonstration Table 14

4.4.6 SSRF WAF Decision Log Analysis

```
(kali㉿kali)-[~/waf-proxy]
$ node waf.js
Local WAF proxy running at http://127.0.0.1:8080
Forwarding allowed traffic to Juice Shop at http://127.0.0.1:3000
XSS test endpoint: http://127.0.0.1:8080/waf-search?q=PAYLOAD
SSRF protected endpoint: POST http://127.0.0.1:8080/profile/image/url
(node:778661) [DEP0060] DeprecationWarning: The `util._extend` API is de
recated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was create
d)
[SSRF WAF BLOCKED] Pattern: 127.0.0.1 | imageUrl: http://127.0.0.1:9090/
test.jpg
[SSRF WAF ALLOWED] imageUrl: http://2130706433:9090/test.jpg
```

Alternative localhost format allowed

Literal localhost URL blocked

Figure 20: Exploitation through WAF Evasion – SSRF WAF Decision Log

Field	Details
Test Activity	SSRF WAF evaluation
Target Endpoint	/profile/image/url
Blocked URL	<u>http://127.0.0.1:9090/test.jpg</u>
Allowed URL	<u>http://2130706433:9090/test.jpg</u>
WAF Result	Blocked (literal localhost) / Allowed (numeric localhost)
Application Response	302 Found redirect to /profile
Listener Evidence	GET /test.jpg received for numeric localhost

Evasion Technique	Alternative numeric localhost representation
Interpretation	The WAF blocked the literal localhost string but failed to detect the numeric representation, which still resolved to the internal server.

Table 15: Demonstration Table 15

4.5 Post-Exploitation

4.5.1 Direct Internal API Request Blocked

```

(kali@kali)-[~]
$ curl -i -X POST "http://127.0.0.1:8080/profile/image/url" \
-H "Content-Type: application/x-www-form-urlencoded" \
-b "token=$TOKEN" \
--data-urlencode "imageUrl=http://127.0.0.1:9100/internal/admin"
HTTP/1.1 403 Forbidden
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 166
ETag: W/"a6-QjPZskMoykuRGBWf00esfoLyeaw"
Date: Sat, 02 May 2026 05:50:59 GMT
Connection: keep-alive
Keep-Alive: timeout=5

<h1>403 Forbidden</h1>
<p>Request blocked by local WAF proxy.</p>
<p>Detected pattern: 127.0.0.1</p>
<p>Reason: SSRF internal address detected</p>

```

Direct internal API URL submitted

Request blocked by WAF

Internal address detected

Figure 21: Post-Exploitation: Direct Internal API Request Blocked

Field	Details
Impact Tested	Internal API access attempt
Target Feature	Profile image URL upload
Submitted URL	<u>http://127.0.0.1:9100/internal/admin</u>
WAF Result	Blocked
Application Response	HTTP 403 Forbidden
Detection Pattern	127.0.0.1
Key Observation	The WAF detected and blocked direct access to the internal API endpoint
Interpretation	Direct localhost-based SSRF requests to internal services were successfully prevented by the WAF

Table 16: Demonstration Table 16

4.5.2 Internal API Access via SSRF WAF Bypass

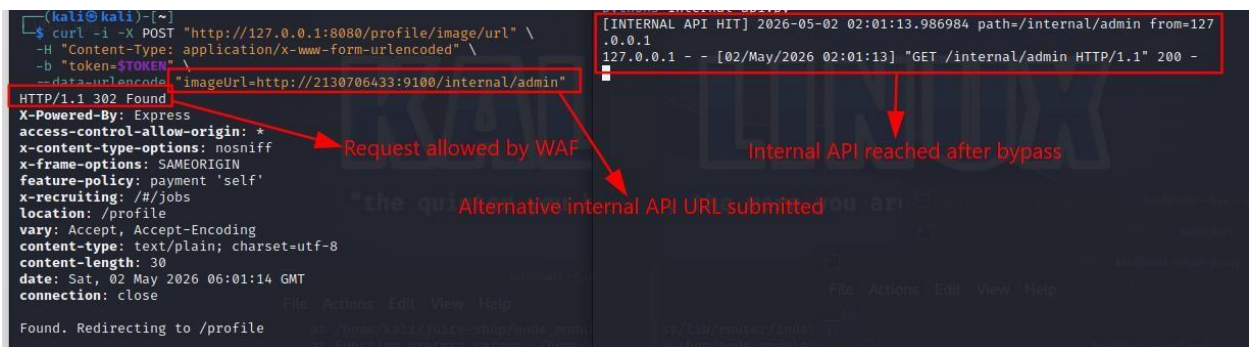


Figure 22: Post-Exploitation: Internal API Access via SSRF WAF Bypass

Field	Details
Impact Tested	Unauthorized internal API access
Target Feature	Profile image URL upload
Bypass URL	http://2130706433:9100/internal/admin
WAF Result	Allowed
Application Response	302 Found
Internal API Evidence	GET /internal/admin HTTP/1.1" 200
Interpretation	The SSRF bypass allowed the application server to reach an internal-only service.

Table 17: Demonstration Table 17

5 Legal Implication of the attack in relation to ETA 2063

The Electronic Transaction Act (ETA) 2063 provides Nepal's legal framework for digital systems and cybersecurity, defining the boundaries for lawful ethical hacking and penetration testing (Nepal, 2026).

Legal Constraints

ETA 2063 criminalizes unauthorized access to computer systems and prohibits activities such as data tampering or service disruption (e.g., XSS, SSRF, SQL injection) unless performed with proper authorization. Violations can result in fines or imprisonment depending on severity.

Ethical Hacking Implications

Ethical hacking is legal only when conducted in authorized, controlled environments such as OWASP Juice Shop or under explicit permission. The law also requires full documentation of testing activities and strict adherence to the defined scope to ensure accountability and legal compliance.

Practical Impact on This Report

All testing in this study was conducted in a controlled local environment, ensuring compliance with ETA 2063. This separation from production systems allowed secure evaluation of vulnerabilities without legal violation.

Recommendations

Organizations should formalize authorization processes, maintain detailed documentation of security testing, and ensure all ethical hacking activities remain within defined scope to align with ETA 2063 requirements.

6. Conclusion

This report investigated how Cross-Site Scripting (XSS) and Server-Side Request Forgery (SSRF) attacks interact with Web Application Firewall (WAF) protections in a controlled OWASP Juice Shop environment. Ethical hacking methodologies, including reconnaissance, vulnerability identification, exploitation, and post-exploitation analysis, were used to evaluate the effectiveness of WAF filtering mechanisms.

The investigation showed that the WAF successfully blocked several standard attack payloads. However, modified XSS payloads and alternative SSRF localhost representations were able to bypass signature-based filtering and execute successfully. These results demonstrate that although WAFs provide an important layer of defense, static filtering mechanisms alone may not be sufficient against adaptive attack techniques.

The study also highlighted how successful WAF evasion can increase security impact by allowing access to internal resources through SSRF techniques. This reinforces the importance of layered web application security, including secure coding practices, proper input validation, output encoding, continuous monitoring, and regular penetration testing.

All testing activities were conducted within a controlled local environment using OWASP Juice Shop in compliance with the Electronic Transaction Act (ETA) 2063 of Nepal, ensuring that the investigation remained within legal and ethical boundaries.

7. References

cloudflare, 2026. *cloudflare.com*. [Online]

Available at: <https://www.cloudflare.com/learning/ddos/glossary/web-application-firewall-waf/>

[Accessed 10 May 2026].

Coursera Staff, 2026. *coursera.org*. [Online]

Available at: <https://www.coursera.org/articles/what-is-ethical-hacking?msocid=1ed7ee067c3d67710476fbe77dcf666e>

[Accessed 14 March 2026].

Desai, K., 2026. *bugbase.ai*. [Online]

Available at: <https://bugbase.ai/blog/top-10-ways-to-bypass-waf>

[Accessed 18 October 2024].

Finn, A., 2026. *petri.com*. [Online]

Available at: <https://petri.com/web-application-firewall/>

[Accessed 25 March 2025].

geeksforgeeks, 2026. *geeksforgeeks*. [Online]

Available at: <https://www.geeksforgeeks.org/ethical-hacking/5-phases-hacking/>

[Accessed 10 March 2026].

indusface, 2026. *indusface.com*. [Online]

Available at: <https://www.indusface.com/learning/types-of-waf/>

[Accessed 1 May 2026].

Khandelwal, S., 2026. *thehackernews.com*. [Online]

Available at: <https://thehackernews.com/2018/09/british-airways-data-breach.html>

[Accessed 07 September 2018].

Nepal, N., 2026. *notarynepal.com*. [Online]

Available at: <https://notarynepal.com/blog/electronic-transactions-act-nepal>

[Accessed 20 May 2026].

Sharma, V., 2026. *knowledgehut.com*. [Online]

Available at: <https://www.knowledgehut.com/blog/security/types-of-ethical-hacking>

[Accessed 11 August 2022].

Stallings, W. & B. L., 2018. *Computer Security: Principles and Practice*. 4th ed ed. New York: Pearson.

8. Appendix

8.1 Advantages and Limitations of Ethical Hacking

Advantages of Ethical Hacking

- Proactively identifies and remediates vulnerabilities before exploitation.
- Reduces the risk of data breaches, financial loss, and reputational damage.
- Provides realistic assessment of system security under attack scenarios.
- Improves overall security posture, resilience, and incident response readiness

Limitations of Ethical Hacking

- Requires specialized expertise and resources.
- Cannot guarantee detection of all vulnerabilities, including zero-day threats.
- May be time-consuming and operationally complex in large environments.
- Effectiveness depends on the defined scope, tools, and tester skill level.

[Click here to go back to top](#)

8.2 Advantages and limitations of WAF

Advantages of WAFs:

- Blocks common web attacks, including SQL Injection and XSS.
- Filters malicious traffic proactively.
- Provides real-time monitoring, logging, and alerts.
- Supports rate limiting, IP blocking, and geofencing.
- Strengthens security as part of a defense-in-depth strategy.

Limitations of WAFs:

- Can be bypassed with sophisticated or obfuscated payloads.
- May generate false positives that block legitimate traffic.

- Limited understanding of complex application logic.
- Cannot guarantee detection of novel attacks without tuning.
- Requires ongoing updates and rule maintenance.

[Click here to go back to top](#)

8.3 Phases of Ethical Hacking

Reconnaissance (Information Gathering): It is the first phase, where information about the target is collected using passive methods (such as OSINT and public data) and active methods (such as DNS queries). The objective is to understand the target environment and identify the attack surface.

Scanning and Enumeration: It involves analyzing the collected data to identify live hosts, open ports, services, and potential vulnerabilities. This phase includes network mapping, vulnerability scanning, and extraction of system details such as users and resources.

Gaining Access (Exploitation): It focuses on exploiting identified vulnerabilities to gain access to the system. Techniques may include password attacks, software exploitation, and web-based attacks such as SQL injection. This phase demonstrates how an attacker could compromise the system.

Maintaining Access (Post-Exploitation): It evaluates whether access can be sustained and expanded. It includes privilege escalation and limited lateral movement to assess the impact of a breach. In ethical hacking, this is performed in a controlled manner without creating permanent access.

Covering Tracks (Attacker Perspective): It is a phase in malicious hacking where attackers attempt to hide their presence by deleting logs or obscuring activity. However, in ethical hacking, this is not performed; instead, all actions are documented and reported transparently.

Reporting and Remediation: It is the final phase, where all findings are documented, risks are analyzed, and recommendations are provided to fix vulnerabilities. This ensures transparency and helps organizations improve their security posture.

Additionally, ethical hackers perform cleanup and restoration, removing any test artifacts and ensuring systems are returned to their original state. Unlike malicious hacking, no attempt is made to hide activity, and all actions are fully documented.

[Click here to go back to top](#)

8.4 Methodology for WAF Evasion

This study follows a structured ethical hacking methodology designed to simulate how attackers interact with web applications protected by Web Application Firewalls (WAFs). Rather than relying on a single payload or attack attempt, the investigation uses an iterative testing process in which inputs are submitted, system responses are analyzed, and payloads are modified to evaluate how filtering mechanisms behave under different conditions.

The methodology focuses on identifying how the application processes user-controlled input, how the WAF detects malicious patterns, and how modified payloads can influence filtering outcomes. Testing was conducted in a controlled local laboratory environment to ensure safe and authorized experimentation.

8.4.1 Tools and Environment

The practical investigation was performed using commonly used penetration testing tools and a locally hosted vulnerable web application.

Tools and environment used:

- Kali Linux – used as the primary penetration testing environment to simulate an attacker system
- OWASP Juice Shop – deployed locally as the target web application for vulnerability testing.

- Web Browser (Google Chrome / Firefox) – used to interact with the application and observe application behavior
- Browser Developer Tools – used to inspect page structure, monitor HTTP requests and responses, and analyze script execution
- Python HTTP Server – used during SSRF testing to verify whether server-side requests were successfully generated

The OWASP Juice Shop application and supporting services were hosted locally within the testing environment. This ensured that all experiments involving XSS, SSRF, payload execution, and WAF evasion were performed safely without affecting external systems or production environments.

8.4.2 Techniques Applied

The following techniques were applied during the investigation to evaluate WAF behavior and web application security controls:

- Cross-Site Scripting (XSS) payload testing against user-controlled input fields
- Server-Side Request Forgery (SSRF) testing using externally supplied URLs
- Baseline payload testing to identify WAF filtering behavior
- Payload modification and structural variation to evaluate signature-based detection weaknesses
- Obfuscation and alternative input representation techniques for WAF evasion
- Context-aware testing across different application inputs and execution contexts
- Iterative testing based on observed application and WAF responses
- Monitoring of HTTP requests, responses, and server-side interactions during exploitation attempts

These techniques allowed the investigation to assess both the effectiveness and the limitations of WAF protections against adaptive input-based attacks.

[Click here to go back to top](#)

8.5 Analysis Based on Penetration Testing Methodology

1. Information Gathering (Reconnaissance)

The attackers appear to have focused specifically on the payment processing functionality of the British Airways website. The targeted nature of the attack suggests prior identification of critical data entry points where sensitive information is handled. This indicates a level of reconnaissance aimed at understanding the application's structure and workflow.

2. Enumeration

The attack implies that the attackers analyzed how the web application handled client-side scripts and input processing. The ability to inject malicious JavaScript suggests that weaknesses existed in script validation, integrity controls, or third-party resource management, which were likely identified during this phase.

3. Exploitation (Gaining Access)

The breach was executed through the injection of malicious JavaScript code into the payment page. This code captured user input, including payment details, at the point of entry and transmitted it to an external server. This represents exploitation at the application layer, without requiring direct compromise of backend infrastructure.

4. Post-Exploitation

Once deployed, the malicious script remained active and continued to collect customer data over a period of time. This indicates that the attackers were able to maintain their presence within the application environment, suggesting insufficient monitoring or detection mechanisms.

5. Persistence and Stealth (Attacker Behavior)

Although not formally documented as a separate phase, the attack demonstrates characteristics of persistence and stealth. The injected code operated in a manner that avoided immediate detection, likely by blending with legitimate scripts. This behavior aligns with what is commonly referred to as covering tracks in malicious contexts. It is

important to note that such actions are not part of ethical hacking practices, where all activities are conducted transparently.

6. Reporting and Remediation (Defensive Response)

Following the discovery of the breach, British Airways removed the malicious code, notified affected users, and implemented additional security measures. The organization was subsequently fined £20 million under GDPR, reflecting deficiencies in security controls and incident response.

[Click here to go back to the top](#)

8.6 Legal compliance mapping table

ETA 2063 Provision	Relation to Ethical Hacking Activities	Application in This Investigation	Legal Implication
Unauthorized Access Provisions	Restricts access to computer systems without authorization	XSS and SSRF testing were conducted only within a locally controlled OWASP Juice Shop environment	Ensures that testing activities remain legally authorized and do not violate cybercrime provisions
Data Interference and Misuse Provisions	Prohibits unauthorized modification or misuse of digital systems and data	Payload execution and SSRF testing were limited to isolated laboratory systems	Prevents legal liability associated with affecting production or third-party systems
Electronic Communication and Network Monitoring Provisions	Regulates interception and observation of digital communications	HTTP requests and responses were analyzed only within the local testing environment	Maintains compliance by avoiding unauthorized monitoring of external systems
Cybercrime Penalty Provisions	Defines legal consequences for unauthorized cyber activities	Scope limitations and controlled testing procedures were maintained	Reinforces the importance of authorization, accountability, and ethical conduct

		throughout the investigation	
Documentation and Accountability Requirements	Supports evidence collection and responsible reporting practices	All payloads, WAF responses, and testing outcomes were documented within the report	Demonstrates transparency, accountability, and ethical compliance during testing

Table 18: Legal compliance mapping table

8.7 Payload comparison table

Attempt	Payload	Purpose	Result	Interpretation
1	<script>alert(1)</script>	Baseline XSS testing	Not executed	Direct script-tag execution was ineffective within the search context
2		Modified payload testing	Executed successfully	Alternative HTML tag and event-handler structure successfully triggered JavaScript execution
3		Page context confirmation	Executed successfully	Confirmed that execution occurred within the Juice Shop application domain

Table 19: Payload comparison table

[Click here to go back to top](#)

8.8 Design

The practical work was designed around a controlled web application security testing environment to evaluate how Web Application Firewalls (WAFs) respond to input-based attacks. OWASP Juice Shop was selected as the target application because it intentionally contains web vulnerabilities suitable for ethical hacking demonstrations. A local WAF filtering mechanism was implemented to inspect incoming requests and block suspicious payloads based on predefined rules. The design focused on testing Cross-Site Scripting (XSS) and Server-Side Request Forgery (SSRF) attacks under different input conditions to observe how payload structure and representation influence WAF behavior.

8.9 Implementation

The environment was implemented using Kali Linux as the penetration testing platform and OWASP Juice Shop as the locally hosted target application. Different payloads were submitted through user-controlled input fields, including the search function and profile image URL feature. Browser Developer Tools and Burp Suite were used to inspect and modify HTTP requests during testing. Standard XSS and SSRF payloads were first tested to establish baseline WAF behavior, followed by modified and obfuscated payloads designed to evade filtering mechanisms. A Python HTTP listener was also configured to confirm successful SSRF requests during exploitation testing.

8.10 Testing

Testing was conducted systematically across multiple stages, including reconnaissance, vulnerability identification, exploitation, and post-exploitation analysis. Baseline payloads such as `<script>alert(1)</script>` and localhost SSRF requests were initially tested to determine whether the WAF could detect and block common attack patterns. Additional payloads using modified event handlers, alternative structures, and numeric localhost representations were then applied to evaluate WAF evasion capability. HTTP responses, browser behavior, WAF logs, and server-side request evidence were monitored throughout the testing process to confirm attack execution and filtering outcomes.

8.11 Evaluation of Results

The results demonstrated that the WAF successfully blocked several direct and commonly recognized attack payloads. However, modified XSS payloads and alternative

SSRF localhost representations were able to bypass filtering rules and execute successfully within the application environment. The investigation showed that signature-based WAF protections can be ineffective against adaptive payload techniques that alter structure without changing functionality. The practical work therefore highlighted both the strengths and limitations of WAF-based defenses and emphasized the importance of layered security controls, secure input handling, request normalization, and continuous security testing to improve web application protection.